

PRD

Parity Release Testing Pipeline PRD

Authors: Andrei Sandu Oliver Tale-Yazdi Javier Viola Villanueva Mak Hlukhovtsov

<PLEASE ADD YOURSELF>

LGTM: Adrian-Costin Catangiu

Status: in review

Last updated: 28 Aug 2026

Name	Approved	Comments
Adrian-Costin Catangiu	✓	
Karim Jedda	✓	Barring the small suggestion below, it is LGTM
Joe Petrowski	✓	Approving the PRD, but I still think the high level tabs could be cleaned up. There should be some concrete north star metrics included, e.g. upgrade to Polkadot every 4 weeks.
Torsten Stüber	✓	This project with all its requirements is quite ambitious and it will bring an undeniably strong value to Parity engineering. The requirements listed below are over shooting in my opinion and it would take quite a lot of work to get all of that implemented. The Q3 MVP plan and the milestones are already a more attainable reduction but even then I would first start with something much simpler (to get the ball rolling): focus only on

		PreviewNet first with nightly fully automated upgrades and make the QA Gates design for all relevant components work. This doesn't even require the EM user interface or just a very simplified version of it.
--	--	--

TL:DR

As of now, releases (especially in preproduction environments) are characterized by manual and shallow testing that lacks essential automation, long-duration checks, application-level verification, poor/incoherent/inconsistent communication to stakeholders, and inconsistent gating between different networks/environments. These deficiencies slow down Parity's progress, increase operational costs, and elevate the risk of shipping bugs to users.

We propose creating a unified self-service platform for robust releases, for both upgrading existing environments as well as spinning up ephemeral environments — e.g. for local or long running tests as well as demos. The set of tools and processes developed will enable a regular cadence releases, of quality commensurate to each network.

Problem statement

1. The release process is
 - a. too slow/delayed, instead of at a regular cadence (progressively per network all the way to production), and
 - b. of inconsistent quality, instead of consistently meeting the quality bar (of the respective network).
2. The release process does not make features available to Platform and Applied developers in a clear (i.e. what configuration on what environment) nor timely (e.g. nightly) manner.
3. Creating and managing ephemeral environments (e.g. for long term testing or showcasing) is manual, error prone, and slow.

Requirements

Functional requirements and features

Short term (2026):

- **Fully self service:** Developers deploy/manage custom test networks without requiring help from the SRE team. Any developer can add new components, tests, and testing pipelines via github PRs.
 - ICs can spin up ephemeral network instances — on demand, end-to-end testing, long running test networks and showcase networks.
- **Zombienet / Omni tester SDK:** Glue for full integration with the Test Environment Manager and the self service system. Enables easy porting of existing tests.
- **Test Environment Manager:** A centralized dashboard and standardized API for deploying, monitoring, and managing testing environments, including visibility into component health metrics, status logs, and dependency graphs.
- **Full control & observability stack:** Developers get access to SUDO, Prometheus, Grafana and SSH keys.
- **Standardized environment definitions:** Mirror development, test and production — PreviewNet, Paseo-next, Paseo, Westend, Kusama, Polkadot.
 - Enabling automated testing across these all networks including their respective data (whereas we currently don't automatically test against all these networks).
- **Local dev/testing:** Tests can be run locally, for debug purposes (assuming the local machine has enough resources).
- **Automated QA Pipelines:** Mandatory QA checkpoints for release candidate promotion. The tests should explicitly identify whether updates maintain compatibility or introduce breaking modifications.
 - To upgrade environments without accidentally introducing breaking changes when considering the environment as a whole.
 - Target environments: PreviewNet (Q3), Westend, Kusama & Polkadot (Q4)
- **Enable standardised per environment quality requirements:** Each team can define the quality requirements (e.g. testing thoroughness) per environment.
- **Progressive releases across environments:** Teams decide which environment their code/functionality is ready for based on agreed standards for the thoroughness of their test coverage and readiness of their dependencies.
 - Part of this process is to decide this per network, e.g. Polkadot has much stricter requirements than PreviewNet.
- **Automated Environment updates:** Once all tests have passed, the environment is updated with the new versions or components.
- **Github Release integration:** Automation picks Release Candidates and starts the relevant test pipelines.

Long term (2027):

- **Automated live network upgrade:** After the environment is updated with the new versions or components, nodes that Parity runs and testnet runtimes are upgraded automatically.
- **Load & stress testing.**
- **Snapshot capabilities:** Environment instances can be paused when not needed (freeing resources, except storage), and resumed later. Environment instances can also be cloned and shared.
- **Scalable deployment configuration:** Large scale tests can be run (100+ nodes).
- **Automated Release Notifications:** Integrated subscription mechanism enabling stakeholders to receive updates via Matrix, email, or RSS. This service tracks specific component or environment tags, providing timely, automated alerts regarding new releases, system deployments, or configuration modifications.

Non-functional requirements

- **Cost:** The system must optimize for lowest cost by default (picking instance types, toaster, etc), unless instructed otherwise (for performance/stress testing).
- **Usability:** Anyone should be able to get started by starting the front end application. Tutorial mode + Wiki.
- **Scalability:** The system must allow for a large number of concurrently running test environments (100+).
- **Availability:** 99%. The system must use multiple cloud providers and the Toaster to ensure that development activities are not affected by compute/storage provider outages.
- **Development velocity:** Should not be limited by this solution. In fact it should be improved, i.e. this solution should not be a bottleneck.

Enshrined requirements - **Applied engineering**

- Dev and Stable environments
- E2E testing (ZN & Omni tester SDK)
- Identity backend (part of environment)
- Integration into hosts / collaboration with the platform team to find a way how these provisioned environments can work with Trinity.

Target users

Initially the focus of this initiative is restricted to Parity. List of teams that will need to sign off on requirements:

- Applied Engineering function
 - Polkadot App @novasama: Most important component to get QA.
 - Jollity / DIM2 @lonut: High level component that needs e2e testing to work.
 - @Karim: Most of the components will be from this function.
- Node function

- Smoldot @Sebastian K: Performance critical node component, can test our performance gates here.
- Runtime function
 - Staking @Paolo: The staking component needs to have spotless QA. We can use it to check that a large number of tests can be integrated well.
- Platform function
 - Host-SDK @PG: Central in the stack. Our solution will need to work very well for this team.
 - SRE: Release automation @Egor
- Incubation function
 - Toaster David Dorgan : PolkaJAM needs to be able to spin up long running environments on the Toaster.

External Stakeholders

We will only include those in the feedback process after our MVP to not slow us down:

- Node operators (inside and outside of Parity)
 - Validators
 - Collators
- Network operators (inside and outside of Parity)
 - Fellowship
 - Paseo team

Scope

In scope

- All releases (of node, runtime, host, apps, and 3rd party components). Nightly, pre-release, ad-hoc (branch) and stable releases are in scope.
- Fellowship runtime releases.
- All environments (including PolkaJAM and long running test environments).
- All infrastructure, including Toaster.
- Defining structures and processes.

Out of scope

- Test case definitions and pass criteria. Should be defined by component owner teams.

Expected impact

1. Most issues surface during the QA cycle, rather than testnets and production.
2. Close to real world:
 - a. End to end application testing
 - b. Infrastructure upgrade testing

3. Improved quality Node, Runtime, Platform, Applied engineering releases:
 - a. PreviewNet is upgraded on high cadence (nightly)
 - b. Westend no longer breaks on upgrades
 - c. Significant reduction in production chains upgrade blunders

Per Function Reqs

Requirements Per Function

The release process needs to be generic enough to handle the following requirements per function. We require specific release timelines here to avoid scope creep and ensure that a feature that did not make it into the last cutoff will still release soon.

Runtime

The runtime function specifically needs to be able to:

Deploy Polkadot runtimes every four weeks

- Release the SDK every four weeks
- Integrate the SDK into the runtimes within one week
- Update Bulletin & PoP to latest SDK and integrate within less than one week
- This implies that all production testing must conclude with 1-2 days max

Deploy to PreviewNet every night

- Latest released versions of all components
- Automatically upgrades when all checks are green

Deploy to Westend every week

- Latest released versions of all components that were successfully deployed to PreviewNet
- Automatically upgrades when all checks are green

Individuality

The individuality team needs to be able to upgrade within less than one week and conclude testing to ensure that the runtime repo has time to integrate its latest version.

TODO: What about the private / public repo setup? Where does testing happen? Scope / timing?

Bulletin

The bulletin team also has its own repository that sits between the SDK and runtimes. They therefore also need to be able to update within less than one week and conclude testing.

TODO: What about the private / public repo setup? Where does testing happen? Scope / timing?

Encounter Pallets

The encounter pallets are handled by non-Parity Fellows. We can normally rely on their timing but have no introspection on what testing or QA commitments exist. Impact is reduced since only the Kusama Encounter system chain is affected.

Node

Roll out a new validator node version every ?

Roll out a new collator version to one or more system parachains every ?

Applied Engineering

TODO

- How often do you need to release which components?
- What is your commitment for emergency patches? (which components and how fast?)

Platform

TODO

- How often do you need to release which components?
- What is your commitment for emergency patches? (which components and how fast?)

High level proposal

Proposal

Introduce a robust release testing pipeline and improve communication between infrastructure, app teams and the community. Key strategies are a shift toward a self-service model with automated testing, establishing clear ownership and a clear process for promoting release from experimental to production.

The primary focus is creating a robust testing process and infrastructure aimed to solve current testing shortcomings, but also accommodate future requirements. Once this step is complete, the focus becomes maximizing testing coverage (e2e, long duration and stress).

The expected impact is increased release quality with faster delivery (nightly/monthly releases).

Ownership

Component	Owner	Comments
Infrastructure - Hw: cloud, toaster - Observability tooling - Self service app deployment - Environment deployments	SRE team	
Self service software	SRE team + Node	This is a big chunk of work. HW resource abstraction, zombienet tooling, testnet manager, omni tester, etc.
Environment and management	Node	
Testing pipelines	Node	
Integration / tests / failures / fixes	Component owner team	Each team writes and owns their tests.
Release process	TPM ? Node ?	
Communication	TPM ?	

Defining an Environment

An environment is a comprehensive definition of the specific versions of dependencies, and parameters required by a fully functional RC, system chains and apps. By unifying node, chain state, binary and runtime versions, CLI, network topologies, on-chain configurations, platform services such as TURN client, device uniqueness backend, notification service into

a single, cohesive entity. TL;DR: provide a deterministic snapshot of the stack, serving a role similar to Cargo.lock files.

Environments are customizable in terms of scale, parachains, node versions, runtime versions and network properties.

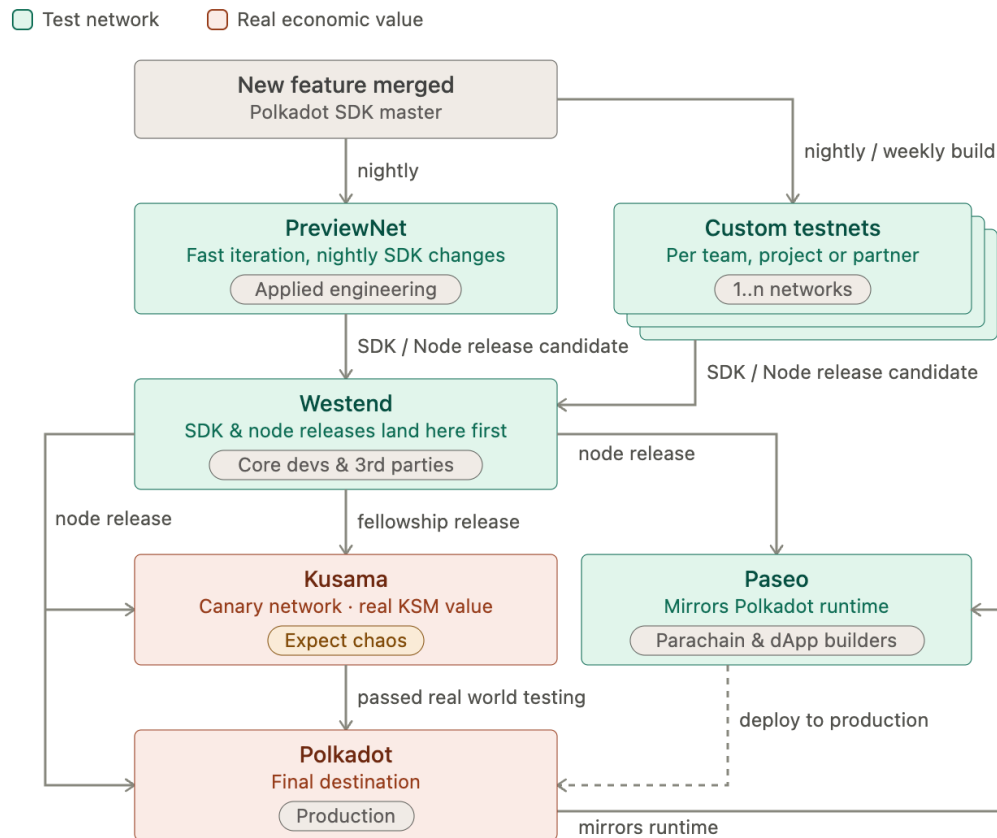
Test Environments

An environment is specified for each current test/prod network. Following a successful run of the QA pipeline, these environments are upgraded prior to the deployment on the live network. This means the environment definition always represents the latest stable state (observed all tests passing).

The original purpose of the live networks remains the same:

- Westend – Parity-maintained Polkadot testnet focused on Polkadot SDK release testing and protocol-level testing. New SDK changes and node releases go here first before hitting Kusama/Polkadot/Paseo. Aimed at core developers and 3rd parties (for example exchanges)
- Paseo – Community-run Polkadot testnet (replaced Rococo in 2024). Mirrors the Polkadot runtime and is the recommended stable environment for parachain and dApp builders to test deployments before going to production (Polkadot).
- Kusama – Polkadot's "canary network." KSM has real economic value, but "expect chaos". Upgrades ship there after Westend but before Polkadot, so it catches issues under real economic conditions.
- PreviewNet – A test network aimed at iterating fast, but it has to provide sufficient stability. Applied engineering is using it. Merged SDK changes should land here nightly.
- Polkadot is the final destination

The path of a feature: from SDK PR to Production



Preview Net

A development and testing environment.

Stable: Paseo RC + system chains

Q: Do we still need this one for App testing ?

Q: Do we need some coordination with the paseo team in terms of runtime releases.

Production: Polkadot/Kusama RC + system chains

Used for testing:

- New Polkadot SDK node releases
- New Fellowship runtime releases
- Combination of node versions (old + new)?
- Application releases end to end testing.
 - Does my new app version work with existing infrastructure ?
 - Do new node/runtime releases break existing apps ?
- Application Frameworks
 - Does my framework work with the new node version?
 - Does the app work with my new framework version?

Next stable (Westend)

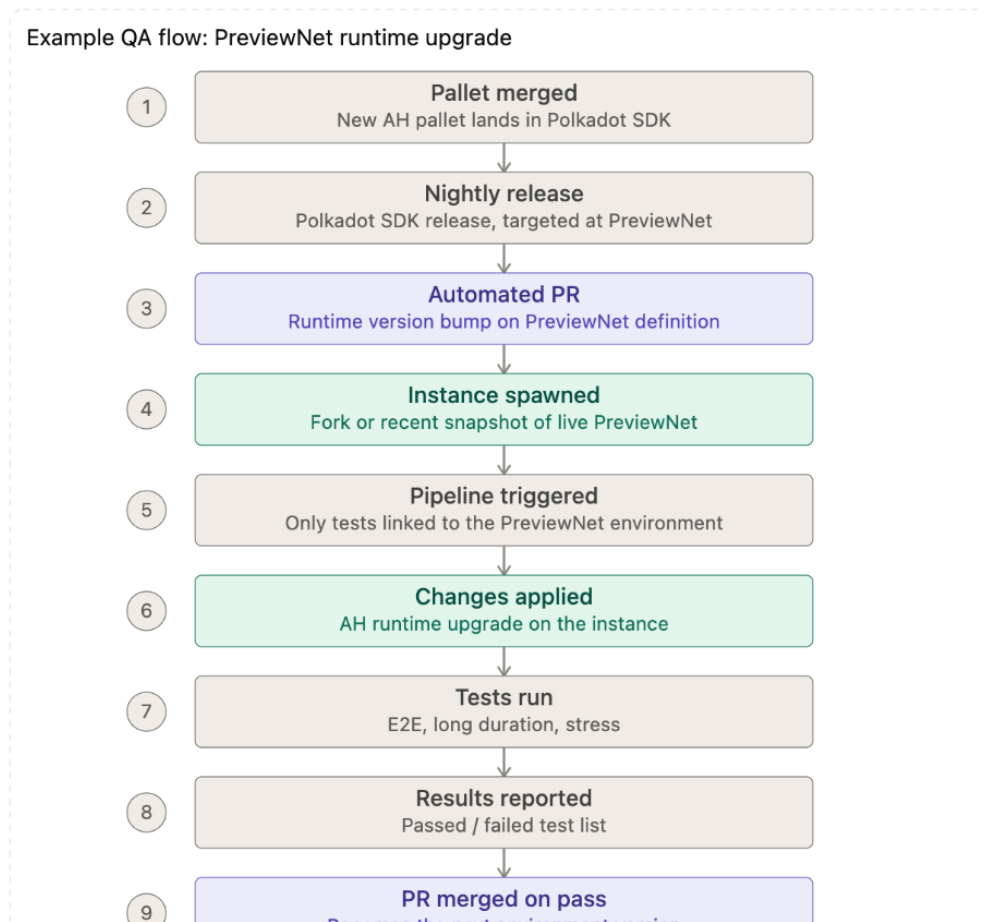
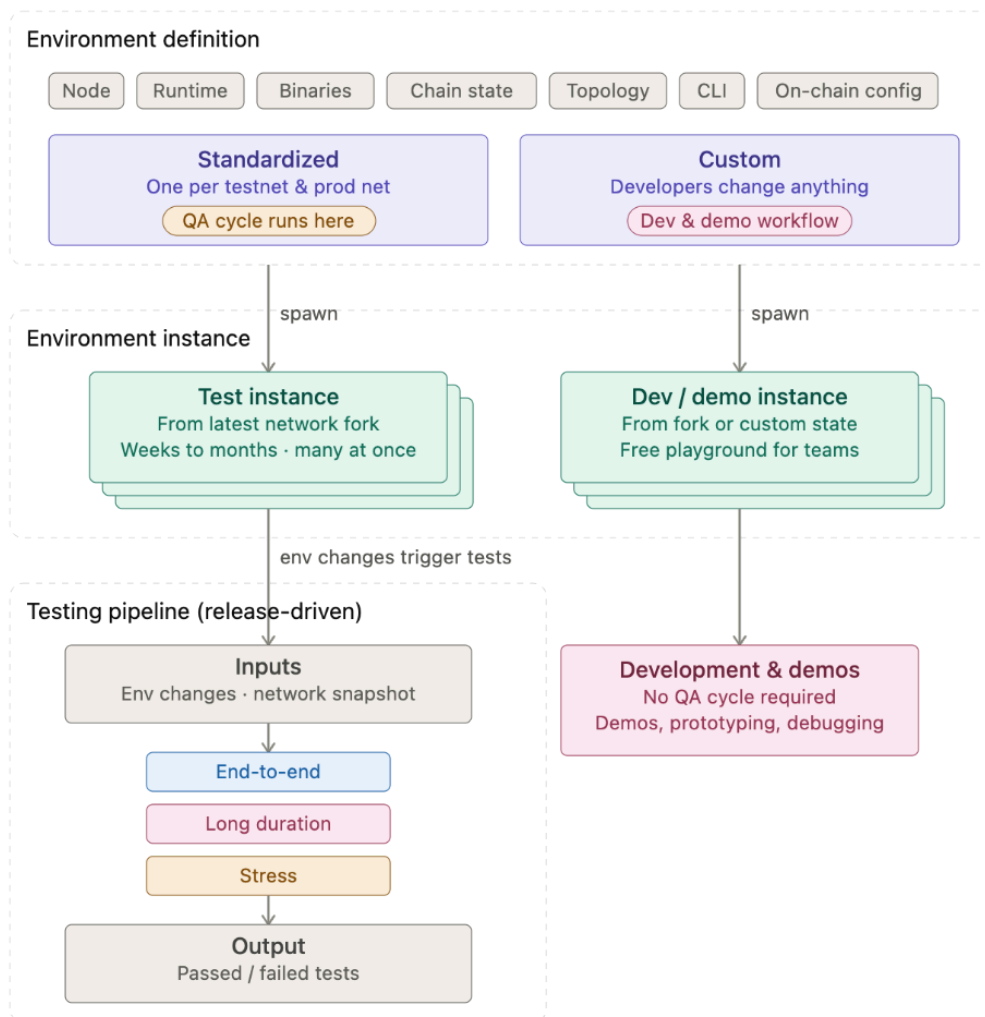
Used for testing:

- Polkadot SDK releases (nodes + runtimes)
- Load/performance
 - Does the new infrastructure work under stress ?
 - How is app UX affected under load ?
- Basic application end to end testing of current prod released app versions
 - Does the new infra update break the next versions of apps ?

Environments are maintained in a public Github repository and are updated regularly only after the changed components have passed all tests. However, the environments can be also updated manually when the need arises.

To facilitate automated ticket creation within the appropriate repositories, every environment includes metadata that specifies the component's development repository. This data allows the release testing pipeline to accurately identify the responsible owner team for each component.

Testing Pipeline Mechanics



Release testing pipeline

The input of the testing pipeline is represented by release candidates. This means any release of nodes, runtimes, applications or configuration changes. The input either adds new components into an existing test environment or updates the versions of existing ones. It can also just be a configuration change (ex: give extra 2 cores to a chain).

The pipeline purpose is to not only verify if the newly released versions pass the tests, but also if all the other components in the stack pass the test.

If all the tests pass, then the change can be applied on the live network. And also the environment definition can be updated to the new versions of the components (automated PR). Initially, owner teams of the affected components need to approve the PR or potentially include additional information, but later, this will be fully automated.

Each environment update PR includes **Release Notes** which bundles the release notes of all components that have been changed, a summary of the version bumps, configuration and any CLI param changes. This will be important to communicate further to operators and the community.

Test management

Test Results and Issue Tracking

Every pipeline run will generate test results, which are converted into corresponding tickets. In the event of a test failure, a ticket is automatically created within the relevant pipeline repository and assigned to the owning team (TL) for resolution.

Release Tracking and Triggers

While pipelines can be executed on demand or scheduled for periodic runs, the most critical executions are those automatically triggered by application and infrastructure releases.

A separate Release GitHub repository will be established to facilitate high-level tracking of Infrastructure and Application releases. Each release will be associated with a dedicated tracking ticket that consolidates the current status across all relevant pipelines and references any outstanding issues.

Crucially, the release cannot proceed until all associated issues have been fully resolved.

Pipeline running

Each pipeline can spawn multiple test environment instances in parallel and multiple tests can target the same test environment instance.

Communication

Releases (stable/unstable/hotfix/etc) will use the aggregated **Release Notes** which should provide sufficient information to both Parity teams and the community.

Links to technical or other resources (*Optional*)

- [Parity's development and testing strategy](#)
- [Product Previewnet V2](#)

Risks

- Running the new system increases GCP cost (mitigate via Toaster resources)
- Test flakiness (mitigate by early focus on robustness and frequent continuous runs)

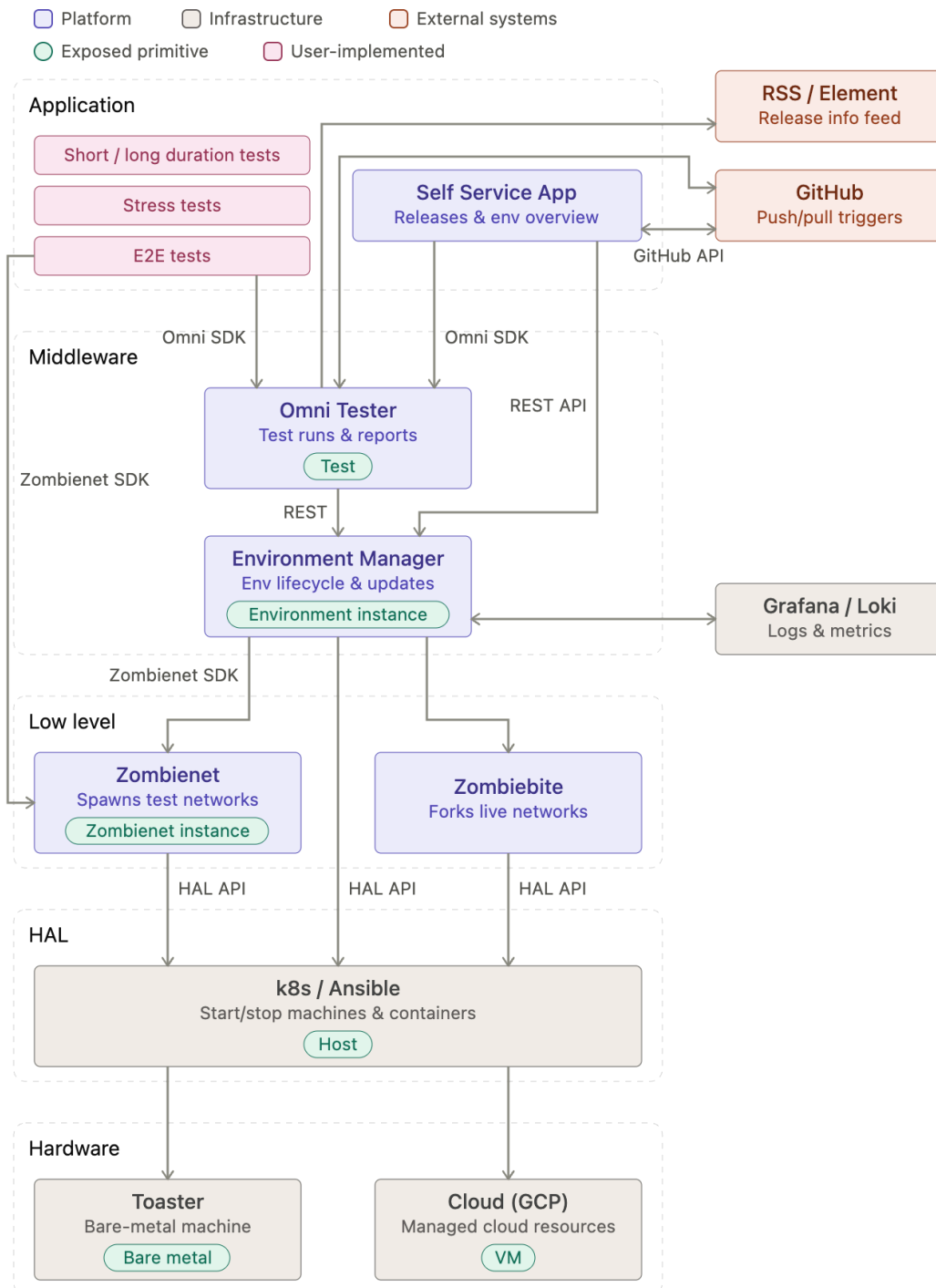
Milestones / Roadmap

See [Implementation plan](#) tab.

High level design

Overview

High level overview of the system with the interaction between the components:



Components

Hardware abstraction layer

Abstracts away the specific compute, storage and networking resources. Optimizes for lowest cost by default (picking instance types, toaster, etc), but also optimizes for best performance when needed.

Open questions:

- We already know K8S works for this purpose, but is it the right solution, what else is there ?
- Can we run k8s in Toaster ?
 - Nomad can run docker images in Toaster.

Zombienet / Zombiebite

Orchestrates and manages the lifecycle of test networks including all environment components (nodes, tools, servers): start/pause/resume/snapshot

It is a low level building block, used by the Environment Manager to spawn instances either from genesis, by forking a live network or starting from a snapshot.

Zombienet SDK provides the infrastructure for writing tests and is integrated with Omni Tester.

Environment Manager

Orchestrates and manages the lifecycle of environment definitions and instances.

[Omni Tester](#)

Runs as a separate service, and orchestrates test runs, results and Github integrations (push/pull)

Omni Tester SDK is a high level wrapper on top of ZombieNet SDK tests.

Self Service App

Simple UX for managing environments and instances. Gives observability into test runs, results and ability to easily spawn custom test environments.

Constraints

Instance start time. Optimize for lowest instance startup time.

Cost. Optimize for lowest cost per instance.

Storage. On-chain and off-chain state and node DBs need to be stored in the environment definition. Storage usage must be minimized, or cheap alternatives like Toaster storage should eventually be used. Large environment definitions can increase instance start latency.

Node agnostic. The system must support Polkadot, Cumulus, PolkaJAM and other node implementations. Only the low level components are allowed to be concerned about node implementation specific details, supporting a new type of node should only require adding support in Zombienet.

Scalability. The system should dynamically scale up to 100 concurrent instances.

User flows

Spawning a New Environment Instance

The environment provisioning process is automated through the self-service web application or REST API.

The workflow proceeds as follows:

1. **Initiation:** Users log into the self-service web application, select the target environment, and initiate the spawn process using default configuration settings.
2. **Request Handling:** The application invokes the Environment Manager (EM) REST API to request a new instance with the specified environment and configuration.
3. **Infrastructure Provisioning:** The EM forwards the request to the Hardware Abstraction Layer (HAL) REST API. The HAL selects the most cost-effective hardware pool and triggers the deployment of the required nodes and containers (e.g., Zombienet, Omni Test) as well as shared assets, like a clone of the github repository that contains the tests
4. **Network Orchestration:** Once the machines are operational, Zombienet orchestrates the startup of all nodes, network connectivity, etc.
5. **Monitoring & Access:** The EM continuously monitors the instance, exposing over REST API instance properties such as Health, Usage, Configuration, RPC endpoints, Grafana/Loki monitoring links, chain specifications, SUDO access, and SSH keys.
6. **UX Presentation:** The self-service web application provides real-time status updates and exposes the aggregated instance properties to the user via a streamlined interface.

Manually starting a batch of tests

- **Test Configuration:** Users access the self-service web application to select a specific environment instance and define the testing scope, (test availability varies by environment)

- **Execution Request:** Leveraging the Omni Test SDK, the application triggers the Environment Manager (EM) REST API to initialize Omni Test binary on an available node within the designated environment.
- **Automated Testing:** Omni Test executes the configured suite, verifying assertions and generating result artifacts upon completion.
- **Visibility:** The self-service interface delivers status updates regarding pass/fail status and provides direct access to execution logs.

Polkadot SDK Release

- **RC Publication:** A new Release Candidate happens.
- **Service Triggering:** Automated GitHub integrations notify the Omni Tester service.
- **Instance Initialization:** Omni Tester leverages calls (EM REST API to spawn instances for every relevant environment associated with the release.
- **Infrastructure Provisioning:** The EM routes the request to the HAL REST API, which identifies optimal hardware and deploys necessary nodes, containers (e.g., Zombienet), and testing repository clones.
- **Binary Execution:** For each active instance, the Omni Test service calls the EM REST API to execute the Omni Test binary on designated environment nodes.
- **Automated Testing:** Testing commences across all environments, with the system generating result artifacts during execution.
- **UX Monitoring:** The self-service application delivers real-time status updates regarding the progress of the active Release Candidate run.
- **Validation:** The process confirms that all assertions in the configured suite have been successfully verified.
- **Environment Promotion:** The Omni Test service generates a Pull Request to implement the current Release Candidate across all target environments. Updates RSS, Element.

Q3 MVP

Q3 MVP high level plan

Goals:

- Any developer can spawn a PreviewNet or Westend ephemeral environment
- Basic QA sanity for PreviewNet releases. (manually triggered)
- ~~— Basic QA sanity for Polkadot SDK releases. (manually triggered)~~

Specific milestones are in the [Milestones tab](#).

Tasks and deliverables

Component	Task	Impl ETA	Status	Owner
Meta	Any developer can spawn a PreviewNet or Westend fork in an ephemeral environment	Q3		
	Basic QA sanity for PreviewNet releases. (manually triggered)	Q3		
	Basic QA sanity for Polkadot SDK releases. (manually triggered)	Q3		
Hardware abstraction	Design including Toaster - it could cover storage for environments data			Maksym ... Nikola Djoric
	Dedicated k8s cluster for Q3 MVP?			Maksym ... Nikola Djoric
Zombienet & Environment support	Environment schema, storage format, update mechanism defined.		Done	Javier Viol... Maksym ...
	Environment Github repository. https://github.com/paritytech/testenv		Done	
	PreviewNet and Westend created.	Q3		Maksym ...
	Zombienet starts a RC + system chain from a Github environment definition.	Q3		Gabriel Cr...
	Support to fully spawn PreviewNet (included identity backend)	Q3		Maksym ... Javier Viol...

Component	Task	Impl ETA	Status	Owner
	Identity backend is currently being rewritten.			
	Local/k8s provider support (also works on your laptop)	Q3		Javier Viol...
Zombiebite	Can be used to update the definition of an environment. Depends on the storage format			Javier Viol...
Environment Manager / Omni tester	Create repository and scaffolding. We are building a monolith to move faster, so Omni Tester SDK and service are part of the same binary.		Done	Andrei Sa... Oliver Tal...
	Doc to clarify how instances are spawned (via PR or REST API)			Maksym ...
	Environment Manager MVP design: update tab - Parity Release Pipeline			Nikola Djoric Javier Viol... Maksym ...
	Create/Update environment API (Zombiebite does heavy lifting for the state, Everything else is provided by user input)			Javier Viol...
	Omni tester SDK (Support for configuring PreviewNet env components, Provides SUDO, RPC, logs, metric endpoints, Support for Runtime upgrade) PoC for launching a ZN test on an instance.			Oliver Tal...
	Start/Stop test suite APIs work			
	Test result artifacts			Oliver Tal...
	ZN integration: block time and finality sanity tests that are environment instance agnostic (no hardcoding) work			
	Start/Stop environment APIs work (Overriding default environment configuration is supported)			
	The whole stack runs in k8s and also on dev machine, including all tests			
	Monitoring stack integration			Nikola Djoric

Component	Task	Impl ETA	Status	Owner
	CLI tool to support for all of the implemented REST APIs			Oliver Tal...
Sanity test suite	Create repository			
	Using ZN and Omni test SDK: write basic block time, finality and runtime upgrade tests			
Self service application	Overview of all environment instances and tests running			
	Start/stop environment (with overrides)			
	Start/stop test (input runtime or new node and targeted environments)			

Environments

We initially focus only on the minimal set of environments needed to properly test the Polkadot node and runtimes for testnet deployment. We define the **PreviewNet** and **Westend** environments with a minimal component set and latest versions.

PreviewNet

Purpose	Try all the newest component versions
Stability guarantee	Minimal. Might require frequent updates and fixes.
Update frequency	Nightly
Environment QA gate	Unit tests, Pact tests, e2e tests
Owner	TBD
Repo	

Components and Version, sourced from [preview net v1](#).

Component	Version
Polkadot Node	master
Kubo IPFS	0.39
Bulletin runtime	master

Component	Version
Web3 storage	master
DotNs	Master
Proof of Ink designs	0.1

Westend

Purpose	Rehearsal space, very similar to production.
Stability guarantee	Should be stable and not require out-of-order fixes.
Update frequency	Weekly and manual (as needed)
Environment QA gate	Previewnet deployment, Stress&Performance tests
Owner	TBD.
Repo	

Components TBD. Either use the versions from PreviewNet or fall back to slightly older versions.

Definition of done

Requirements and definition of done for the Q3 MVP

Zombienet & Environment support

Environment schema, storage format, update mechanism defined

- In accordance with the [design tab](#), we need config schemata
- Settle on which config format: JSON/YAML/TOML etc
- Define parsable schema that can be used for validation of config files
- Example: [prdoc schema](#) can be used in VsCode for good DevEx
- Think about versioning and forward compatibility (optional fields, extensions?)
- Initial design should be simple but not lock us out of future extensions
- For persistent storage: we lock in S3 hashes and abstract the KV storage

Concretely:

- Component schema and minimal example component config
- Environment schema and minimal example environment config

TODO: update mechanism? unclear

Environment Github repository

- GitHub repo with permissions open to Parity
- Branch protected, only way to update is through merge request
- README explaining the purpose of the repo and how it fits into the grand picture

PreviewNet and Westend created

- Environment definitions for PreviewNet and Westend
- Latest known-good versions
- The definitions should only be updated until we are sure that the tentative tooling is stable
- The schema of the repository should be one folder per environment, concretely:
 - ./
 - environments/
 - westend/
 - previewnet/
 - README.md
 - (possibly a Justfile for maintenance operations and schema linting&formatting)

Zombienet starts a RC + system chains from a Github environment definition

- Possibility to start a network from an enum variant:
 - github hash, in the form of
`COMMIT(github.com/<org>/<repo>/commit/<commit-hash>)`
 - Local environment definition repo
`PATH(<path-to-environment-folder>)`

The spawn formats are explicitly an URL

Environment Manager / Omni Tester

Omni Tester + EM Mono repo

- One repository to keep dependencies simple and iteration fast
- Code should still be cleanly separated into different crates
- CI must be setup with test, fmt and linting
- Initial development can be done without code review by the owning developer
- Later on we need quality standards like reviews, regression tests and releases

Start/Stop test suite APIs work

- APIs defined in design doc

Test Result Artefacts

- Test results artefacts should be hermetic and lock in all dependencies that were used for reproducibility
-

ZN integration: block time and finality sanity tests

- Sanity test must only depend on environment config and EM API endpoint (to request RPC endpoints)
- Example metrics for the testnets could be:
 - p95 blocktime [10m] ≤ 2.1
 - Max finality lag during last 10 minutes ≤ 6 and not more than >3 for more than 3 blocks within the last minute

Monitoring Stack Integration

- Focus on the most important metrics: block times, finality lag, node uptime
- Should use the proper exporter so we can fix road blocks early

CLI tool to DevEx Support

- CLI tool with oneliner install command mentioned in the README

- Should use common naming schema like docker, kubectl etc are using:
 - `./em up`
 - `--environment-config`: file path to env config like `./environments/westend.toml`
 - `--environment`: well known names either from local config
 - `--provider <local | k8s:URL>`: where the env should be spawned (maybe other options?)
 - `./em ps`
 - Show all running environments, TBD how the state for this is stored
 - `./em ls`
 - Show all known environment configs (also not running ones)
 - `./em info westend` (addressable by ID, Name or config path)
 - Print info on all components
 - `--format=json`
 - Format output in JSON for programmatic use
 - `--component / -c`
 - Only print info for a single component
 - `./em attach -e <EnvId | EnvConfigPath>`
 - Attach to a remote running environment
 - `./em down westend`
- Then similar for the omni tester:
 - `./ot env connect <EM endpoint>`: Connect to Environment Manager
 - `./ot env sanity-check <EM endpoint | env name>` (TBD how local state is stored)
 - Run sanity checks that are explained below
 - `./ot test ps / ./ot test ls <EM endpoint | env name>`
 - Same semantic as above, show all running and all existing tests.
 - `./ot test start <EM endpoint | env name> <test name | test config path>`
 - `--fail-after=<integer>`: fail after this many tests have failed, defaults to 1
 - `--report-format`: set the format of the test report.
 - `./ot test stop <EM endpoint | env name>`: stop the test and produce a partial report
 - `./ot test reproduce <TEST RESULT FILE PATH>`
 - Re-run a test result and assert that:
 - all originally passing tests still pass
 - all failing ones still fail
 - Prints whether a test result could be perfectly reproduced or not.
 - Could accept an `--all-fixed` flag that now treats all failing tests as failures, as help for the developer when fixing the reproduced bug.

Sanity Test Suite

Create Repository

- README and Justfile for maintenance and common CI operations (linting, fmt, tests etc)
- Parity only access for now
-

Basic Tests

- Sanity check that all components are up and respond on their RPC (version query or specific health probe)
- Check all component versions to match the ones that the EM reported
- If applicable: ensure that permissions of the components are configured such that the OT can access them
-

Chain Agnostic Tests

- Block time / Finality test: check ZN metrics from above
- Feature detection for things like sudo pallet, check if Admin key is set correctly
- Check that dev accounts are funded and genesis spec is sane

Self Service Application

Overview of all environment instances and running tests

- This would wrap the `./em ps` and `./ot test ps` cli commands from above

Start / Stop Environments

- This would do the same as `./em up & down`.

Start / Stop Tests

- This would do the same as `./ot test start & stop`.

WIP

Implementation plan

MVP

As a first target, we will aim on the Polkadot Node and Runtimes. Focusing on the Parity-managed testnet should give us a fast iteration cycle. This would accomplish the following:

- Give us a signal on how good the current design is
- Feedback from Applied Function on how useful it is
- Basic setup of all parts before we need to scale them
- Observations on how developers use the system

We can then use these results to either pivot or continue with the current design. It should also help to garner support and resources to proceed.

The basic idea is to build a minimal version of each part and see how they fit together. We can then figure out stable interfaces and deployments. A minimal first version could be:

Environments

We initially focus only on the minimal set of environments needed to properly test the Polkadot node and runtimes for testnet deployment. We define the **PreviewNet** and **Paseo Next V2** environments with a minimal component set and latest versions.

PreviewNet

Purpose	Try all the newest component versions
Stability guarantee	Minimal. Might require frequent updates and fixes.
Update frequency	Nightly
Environment QA gate	Unit tests, Pact tests, e2e tests
Owner	TBD. Mak maybe?
Repo	

Components and Version, sourced from [preview net v1](#).

Component	Version
Polkadot Node	master
Kubo IPFS	0.39
Bulletin runtime	master

Component	Version
Web3 storage	master
DotNs	Master
Proof of Ink designs	0.1

Paseo Next V2

Purpose	Rehearsal space, very similar to production.
Stability guarantee	Should be stable and not require out-of-order fixes.
Update frequency	Weekly and manual (as needed)
Environment QA gate	Previewnet deployment, Stress&Performance tests
Owner	TBD.
Repo	

Components TBD. Either use the versions from PreviewNet or fall back to slightly older versions.

Environment Manager

The most basic version should only focus on PreviewNet and PNV2 but be done in a generic enough way to be extensible for more environments:

- Show deployed components and their versions
- Allow to fork the environments through an API
- Allow to upgrade the deployed components

All functionality must be accessible via API. The UI is secondary but might be done - trivially with help of AI - if time permits.

Omni Tester

Parts of the omni tester:

- GHA to trigger remote E2E, Pact and performance tests (see the example in Design tab)
- Centralized repo with E2E, Pact and performance tests
- Backend server
 - API for GHA actions
 - API to inspect test results
 - Client to connect to Environment Manager
 - Functionality to open github issues on test failures

Design: Release Process

General Release Process

All components of the Polkadot product stack will be tracked by version and compatibility with other components. Components are arranged into Environments that deploy them. Environments might either be long running, short lived or only for local testing.

The release of a component is gated by its *Release QA Gate*. An environment may upgrade (or reset) its components at any time. Stable environments are advised to do so only once the new component version clears its QA Gate.

Releases

The system handles only released components: artifacts in format such as Docker or binary blobs (runtime wasm). For example, even per PR docker images should work.

The following release types are supported:

- Nightly/Weekly automated releases
- Ad-hoc manual releases (pr/branch)
- Monthly stable/patch/hotfix releases

Component

A component is a piece of software that is part of the Polkadot product stack.

Each component must declare its:

- Name, description
- Repo, code-owners
- Dependencies on other components (must be locked to exact versions)
- Spawn config (see below)
- Monitoring (Otel ports, metrics...)

Spawn Config

Components both expose a *local* and a *remote* spawn config.

The *local* config lets us spawn the service on a server and could just be a docker image or similar.

The *remote* config contains instructions on how to connect to a component that runs outside of Parity's control. This is important to ensure that we can integrate community components in a similar way as to locally spawned components.

The release process will also store per component:

- Known versions, tags, artefact hashes, changelogs
- Compatibility matrix for dependency versions
- Compatibility matrix for version upgrades

Environment

An environment is a deployment of the whole or part of the product stack developed by Parity with potential community contributions; from end-user apps down to the node. It consists of *components* that are arranged in a dependency hierarchy. Some example environments would be: Paseo, W3S Summit Net or Polkadot. Initially, we will focus on core components like the node and runtimes, but eventually expand to all other components higher up in the stack.

Environments can be monitored and maintained through the Environment Manager (silly name, TBD). The EM can be attached either to a live running environment or spin up a new instance. The webserver for the frontend is built into the same binary. And there exists a CLI interface that can be used to access all functionality of the EM by handing it an endpoint.

Specification of an environment:

- Versioning information (sequence number and date)
- List of configurable components (all components that can ever be enabled)
- Component instantiations (all components that are enabled)
- Monitoring config
 - Grafana, Prometheus, Log monitoring...
- Upgrade policy
 - Cadence (daily, weekly, none, ...)
 - Blockers (QA gates, human approval)
 - Enactor (automated, manual)
 - Hotswap migration instructions (if the component supports it, must be codified)
- Rollback policy (optional, manual process can be done at any time)
- SLO, Owner, On-Call
- Optional: Retention policy
 - How long past data is stored
 - Where it is stored
- Optional: Scale, Network topology (injected latencies and faults)

Environment Manager

The EM is responsible for either spinning up all configured components or connecting to them remotely, if they are hosted externally. It should expose a dashboard that shows component versions and general health metrics at a glance. Ideally, all known environments are aggregated into an overview like this:



ENVIRONMENT MANAGER

 Environments

 Components

 Feed

> PreviewNet
 v1.3.0-rc.2

v
 Paseo NV2
 v1.2.1

 PURPOSE
 Shared community testnet. Mirrors production topology for integration and end-to-end testing.

 OWNER
 Owned by **Marco**, with **Thomas** on call this week.
Uptime SLO: **99.0%**

 DEPLOYED COMPONENTS (16)

Polkadot Node	v1.17.2
Polkadot SDK	stable2412-2
Individuality	v0.8.4

 API ENDPOINTS (24)

RELAY RPC	wss://relay.paseo-nv2.network:443	public
HUB RPC	wss://hub.paseo-nv2.network:443	public
ETH RPC	https://eth.paseo-nv2.network/rpc	vpn

 EXPLORERS

 Subscan
  Blockscout

 PAPI

 MONITORING

 Grafana
  Prometheus

> Staging RC
 v1.2.0-rc.4

A deployed environment will also have a standardized API to expose:

- IP/Ports of all components (like “/component/list” with fixed JSON schema)
- Monitoring/status page
 - Block times, block confidence, finality, forks
 - Node health (CPU, RAM, Disk IOs, ..)
 - Runtime and node error logs
- Deep links to auxiliaries like Blockscout, PAPI, ..

- Dependency graph for all components

It will most likely also be useful to see all used version of a component and where they are currently deployed:

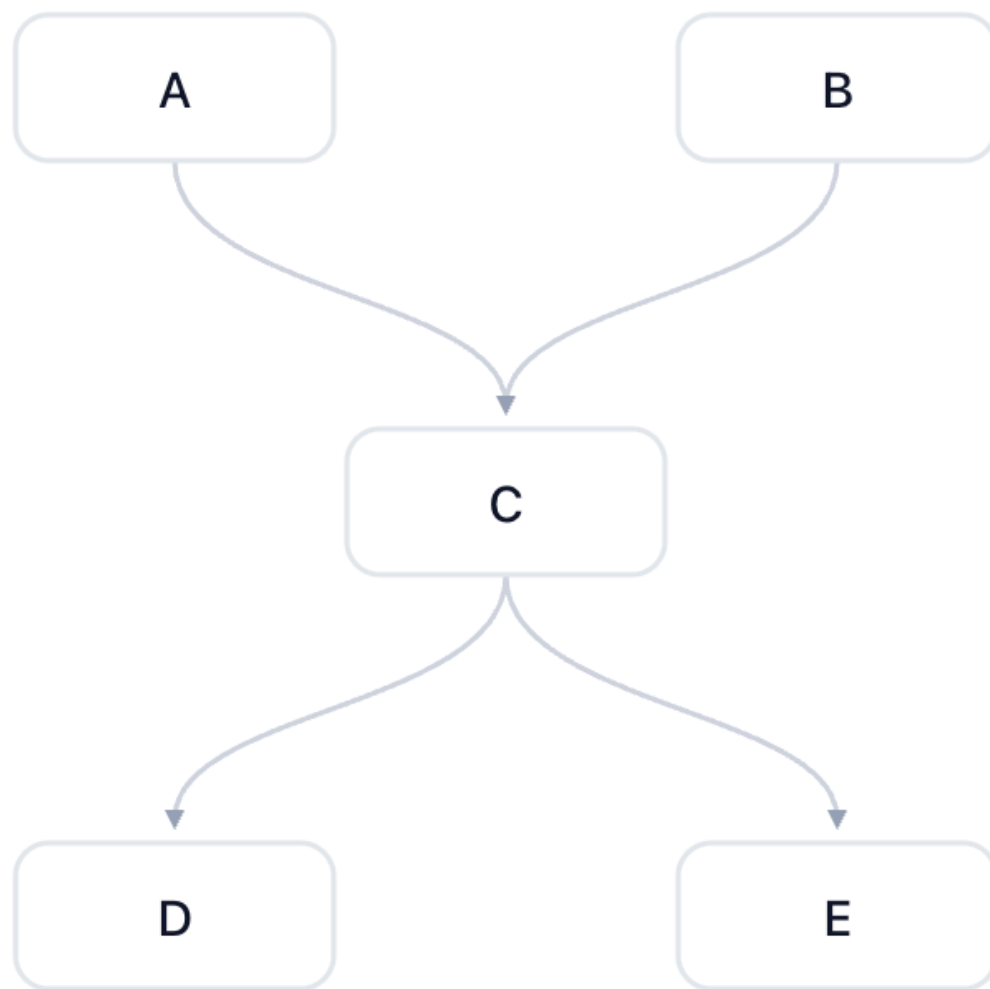
The screenshot shows the 'ENVIRONMENT MANAGER' interface with a dark theme. At the top, there are three tabs: 'Environments', 'Components' (which is active and highlighted in teal), and 'Feed'. Below the tabs, there are four component cards, each representing a different component and its various deployment environments.

Component	Owner	Repository	PreviewNet	Paseo NV2	Staging RC	Mainnet
Polkadot Node	Marco	paritytech/polkadot-sdk	v1.18.0-rc2	v1.17.2	v1.17.0	v1.17.0
Polkadot SDK	Marco	paritytech/polkadot-sdk	stable2503-rc1	stable2412-2	stable2412	stable2412
Individuality	Thomas	example/individuality	v0.9.0-nightly.0612	v0.8.4	v0.8.2	v0.8.2
TruAPI	Thomas	example/truapi	v3.4.0-rc.2	v3.3.1	v3.3.0	v3.3.0

Dependency Graph

All components are arranged in a graph. The edges identify dependencies. In this example, the components A and B could be end-user Apps. C could be something like Trinity and D and E could be node software.

The reason to have a dependency graph is to exploit the fact that we do not need to re-run tests of underlying dependencies when an upstream component changes. This assumption might not always hold, but for high level components we expect it to provide much shorter testing times compared to re-testing the whole stack. The next section demonstrates this.



Full Integration Test

Ensure that post-upgrade versions of multiple components work together as expected. It runs all integration tests of all components.

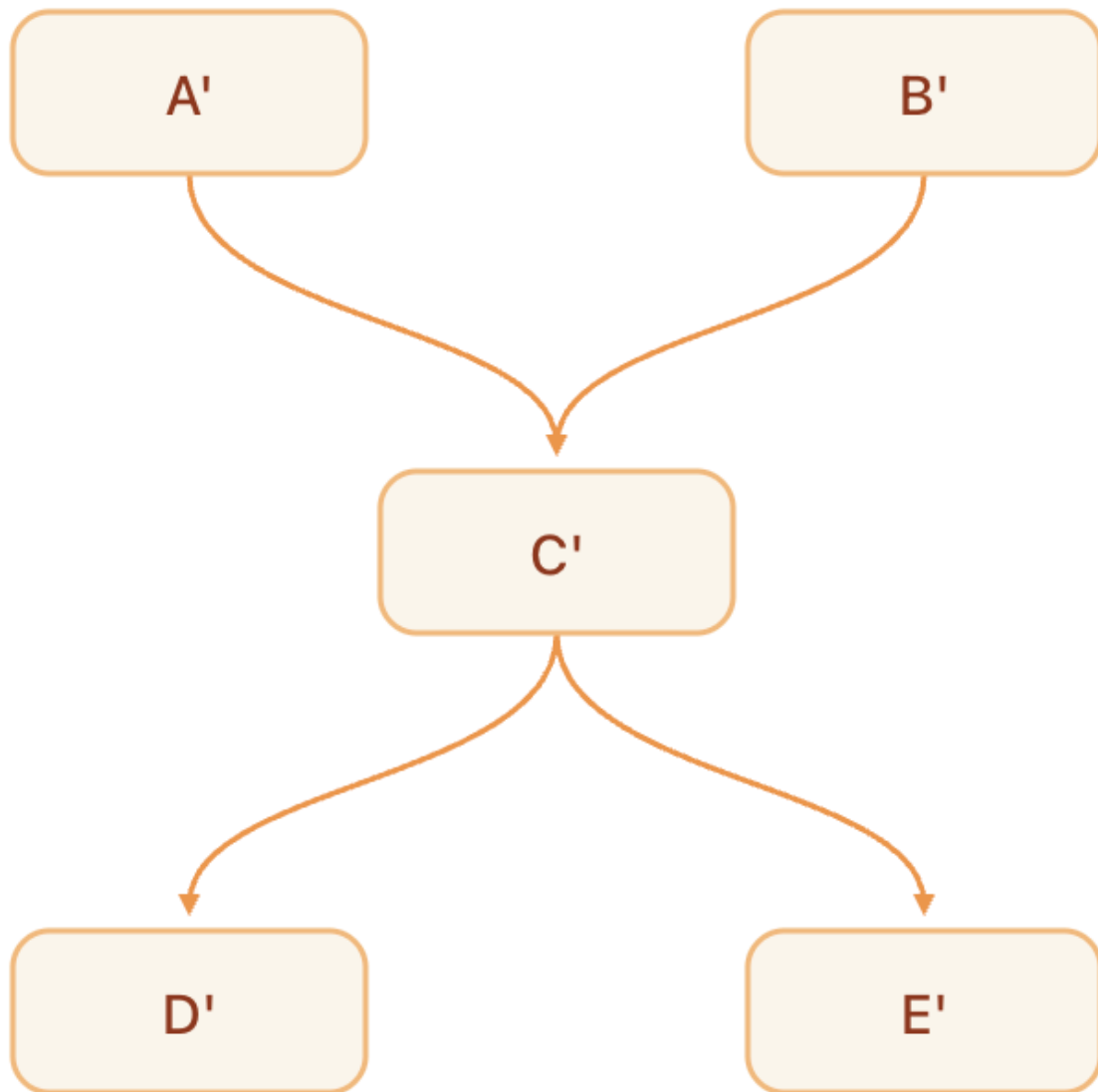
Integration tests solely aim to ensure compatibility with other components (edge in the graph); they do not assert correctness of an individual component.

The output tells us one thing:

- Whether the component configuration (A', B', C', D', E') is good

Execution

- Start the environment with all new versions
- Run component tests
- Mark all components and edges as green



Upgrade Integration Test

Ensure that components can be safely upgraded in a live network.

Tests the hot-swap upgrade path of one or more components. This example tests the version upgrade of component C to C'. All other components run their latest known-good version.

C and all the edges are orange since it is unknown whether the upgrade C -> C' works for all dependents (A, B) and dependencies (D, E).

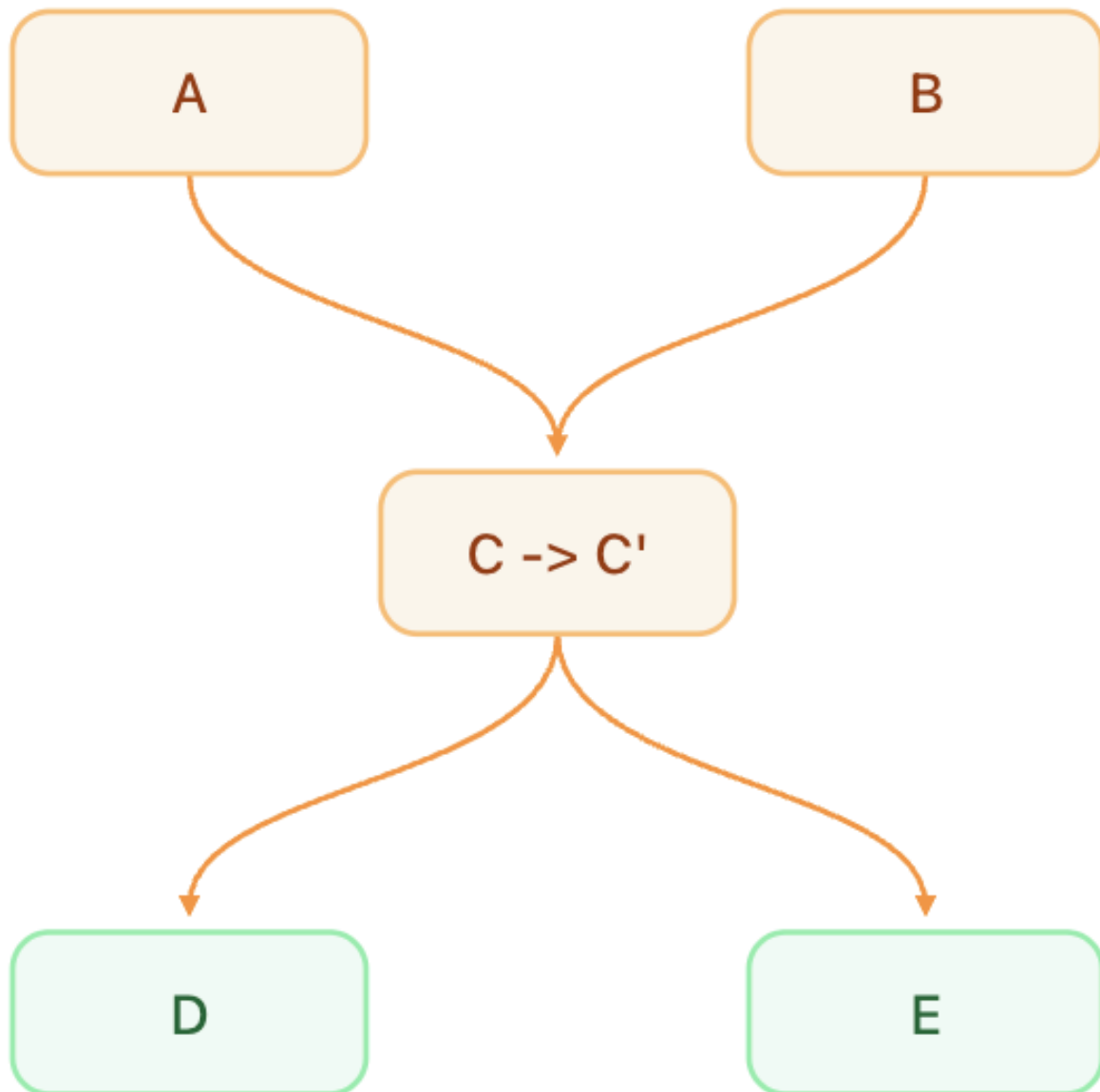
Execution

- Either start the environment with its locked versions or connect to a live deployment
- Run for a while
- Upgrade component C to version C'
- Run post-upgrade check for C'
- Run normal tests for A, B and C'

- Mark $C \rightarrow C'$, C' and all edges as green on success

The output tells us two things under the assumption that (A, B, C, D, E) is compatible:

- Whether the post configuration (A, B, C', D, E) is compatible
- Whether the upgrade $(A, B, C \rightarrow C', D, E)$ is compatible



Stress Test

A *Stress Test* is set up in the same way as an Full *Integration Test* but its tests aim to check for quantitative instead of qualitative requirements.

Such a test might be configured to specifically run:

- For a long duration
- Under adverse conditions (node crashes, network issues, slow nodes, ..)
- With additional workload (many tx, lots of forks, ..)

A stress test tells us the following under the assumption that (A, B, C, D, E) is compatible:

- That (A', B', C', D', E') is compatible under stress

Release QA Gates

Release gates give an indicative YES/NO signal to the component or environment owner on whether a component can be released or an environment be updated to a specific set of versions. The owner might override this signal in case of flaky tests or for other reasons, but in the automated case, it will block a release.

The gate checks would run on Release Candidates, but might also be run earlier or at any convenient time.

Component Release Gate

To assess which component tests need to be run, there is a consequential case-decision to be made. The decision is as follows: if only minor or patch changes were merged since the last release, then the new release is assumed to be *backwards compatible*. Otherwise, it is assumed to be *breaking*. We come to call this the *Backwards Compatibility* of a release.

There is a special case that an emergency fix cannot be implemented as a SemVer-compliant backport. In this case, the component release must be marked as *SemVer violating* and the testing infrastructure will possibly not be able to correctly run the tests until all downstream components are updated to that version.

We run the following tests depending on that against the most bleeding-edge (lowest quality env, like previewnet) environment:

- **backwards compatible**: Upgrade Integration Test
- **breaking**: Full Integration Test

This case split is important to ensure that we can release emergency fix quickly. We only run against the bleeding-edge environment to get a first signal on whether the component release will be most likely deployable on that environment. More stable environments will require tests against their own forked state, see “Environment Release Gate”.

Backwards Compatibility

In order to know whether a component had any breaking changes since the last release, we need a very light form of tracking per change. It can just be a single flag that the developer has to set, for example through a label. It could be worthwhile to use PrDocs in the same way that the SDK uses them, also for the additional advantage of getting an aggregated changelog.

We should most likely employ some AI review tools to spot incorrectly tagged changes since they can cause issues further down in the process.

The backwards compatibility of a release candidate will be set to false as soon as any of the individual changes is *breaking*, otherwise true.

Environment Release Gate

To ensure that one or more components can safely be added, removed, changed or removed from an environment, the environment release gate must pass. In its most basic form, e.g. PreviewNet, it will just be the aggregation of all the component release gates.

For environments that require stability, just the component release gates are not enough. The environment release gate will additionally require that the Upgrade or Full Integration Tests of all components are run against a forked state of itself. The Stress test might reasonably also be added, depending on stability needs.

For production environments, an audit trail of each component should be produced that proves that the component was successfully deployed to each previous environment.

Emergency Fixes

In order to release emergency fixes quickly and at any time, it is important for them to be written in a *backwards compatible* way. Doing a breaking change as an emergency fix generally slows down its rollout, since more tests need to be run. In the worst case, dependent components need to update and be released as well, which would cause an unacceptable slowdown.

Note: We need to explain this constraint to all developers.

Release Comms

Comms should be ideally self-service. The environment manager should allow anyone to subscribe to specific topics and receive notifications over a preferred medium (matrix, email). The following two topics should be offered:

Component Release

The subscription field should allow you to select multiple or all components of interest. A notification will be sent out when a component publishes a new release. It will contain the most high level metadata of the component (name, old & new version, github repo) and the aggregated changelog since the last release with a deep link to the component release page.

Environment Update

The user should be able to select the environments that they are interested in. A notification will be sent out if a component was freshly deployed, upgraded, downgraded or removed from the environment. This information will be automatically gathered and sent out.


ENVIRONMENT MANAGER

 Environments

 Components

 Feed

PreviewNet

Paseo NV2

Staging RC

Mainnet

Subscribe

DATE	COMPONENT	NEW	PREVIOUS
2026-06-02 10:00 · 4w ago	TruAPI	v3.3.1	v3.3.0
2026-06-02 10:00 · 4w ago	Individuality	v0.8.4	v0.8.3
2026-05-26 10:00 · 1mo ago	Polkadot Node	v1.17.2	v1.17.1
2026-05-26 10:00 · 1mo ago	DotNs	v1.1.4	v1.1.3
2026-05-19 10:00 · 1mo ago	IPFS Gateway	v0.33.2	v0.33.1

Manual Bootstrapping

Before the EM can fulfil this role, we will need to have a simpler fan-out mechanism. The simplest way would be to have a `.rss` file on the master branch of the Release repo (to be created) that anyone can add to their RSS feed reader of choice, or use the standard RSS reader Matrix bot to notify them in a channel.

Good release notes are a difficult topic on their own. Realistically, they rely heavily on developers providing good changelog entries for every code or config change. Otherwise it turns into a fancy gargabe-in-garbage-out machine. In the Polkadot SDK it took years to arrive at an acceptable level of developer-provided changelog entries that can be aggregated and useful.

I therefore expect the main part of the challenge here to be about education and discipline.

Omni Tester

The omni tester is the uniform testing framework that components are urged to use. It is kept abstract enough to allow tests to run locally or observe results remotely. The omni tester reports its results to a centralized release repository and might open tickets on serious test failures.

The results here feed into the Release QA Gate of a component.

The omni tester needs to interact with the Environment Manager to spawn ephemeral networks for testing. We can realize this through an API on the EM that accepts (authenticated or VPN only) requests from the Omni tester to spawn, inspect and destroy

environment instances as needed.

This ensures a separation of responsibility and that the EM can be used for other purposes as well - like spawning environments for demos or summits.

Flow

Imagine the TruAPI repository pushes a new commit. They want to ensure that their change did not break anything. The testing procedure goes as follows:

1. The TruAPI CI will produce an artefact of the component (like a .zip file) with the new change included
2. TruAPI CI requests the Omni Tester (OT) to test the new version.
3. OT picks PreviewNet as first environment to test new versions on.
4. OT requests EM to fork the PreviewNet and overwrite the version of the TruAPI component with the artefact from the CI.
5. EM reports spawn success / failure back to OT after doing a simple health check of the environment.
6. OT then runs all the E2E and Pact tests that are expected to succeed.
7. OT observes test results and reports them back to TruAPI CI.
8. TruAPI CI uses this result to decide whether to go ahead with a possibly release or not.

Omni Tests

We split tests into three categories. The latter two will live in a centralized testing repo that will need heavy cross-team collaboration for maintenance.

1. Component unit tests

>"My component does what I expect it to do"

White box tests. Lives in the component repo and maintained by the component developer. This is not directly part of the Omni Tester, but still mentioned here for completeness of the testing strategy.

2. Contract / Pact-style tests

>"Your component does what I expect it to do"

Black box tests. Locks in the observable behaviour. Most likely to be provided by the *consumer* as a form of contract.

3. E2E Tests

>"All components work together end-to-end"

E2E tests that test actual behaviour of components without any mocks. For example: an end-user can play the DIM2 game in the App. This tests the whole stack. Tests are most likely written by developers for their own components.

4. Performance and Long Duration Tests

Performance tests are a form of e2e tests. They configure multiple components at once and act as a validation test, not as a way to debug slow components. The implication is that if there is even one slow component then it can slow down the whole test. However, this is the simpler approach for now as compared to doing dedicated performance tests for each component.

For emergency fixes we need a minimal set of performance tests. Mostly as a smoke screen. This ensures that we can still do some performance validation but are not held up for as long as with a full suite.

Cross-Team Coordination

Our design attempts to minimize the need for cross-team communication. The advantage of this is obvious in any multi-threaded application. The fastest algorithms are *embarrassingly parallel*, meaning they do not need to send data between threads at all.

We try to replicate this through *backwards compatible* patches. Any change that can be done in a backwards compatible way, should definitely be done this way. In the ideal scenario, only the team that suffers from an underlying bug and the team fixing the buggy component need to communicate.

This allows us to patch components in parallel, with the only serializing part being the Omni Tester to order their releases.

Host/App test

Host/App testing vs Runtime testing

We need runtimes not to break apps, and apps to keep working against the runtimes we deploy. Testing that head-on means filling a grid nobody can hold still - so this is a sketch of how we might cut it into two lines instead, plus a third mode that runs no tests at all.

Author: Maksym Hlukhovtsov

Summary

There are two things we need to be confident about:

- that a new chain runtime doesn't break the apps people are already running,
- that our apps and hosts keep working against the runtimes actually deployed.

If we test every combination, we end up filling a grid of every runtime against every host and app - and each axis of that grid belongs to a different team, on its own schedule.

So rather than treating it as one testing problem, it might help to think of it as one environment we configure three ways:

1. Runtime and node testing - substitute the hosts. We supply the host side ourselves, so the chain ends up being the only thing that varies.
2. Host and app testing - fork the stable network once, then run the app tests twice from that same fork: once as it is, once with the runtime upgrade applied. Comparing the two runs is what tells us whether an upgrade affects hosts and apps.
3. Demo mode - spawn the whole stack, with runtime and node binaries overridable by flag, deploy an app into it and show it to someone. No tests, no verdict.

The third isn't really a side effect. It may well be the most useful day to day, and it's part of why this feels worth building properly rather than as test scaffolding.

The problem

These are two different questions, even though they sound similar:

One is that a new chain runtime shouldn't break the apps people already run.

The other is that the apps and hosts we ship should work against the runtimes actually deployed.

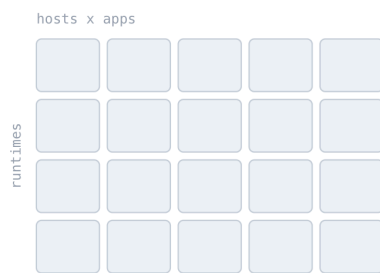
Both are really compatibility questions rather than correctness ones.

If we try to answer them directly, we end up with a grid: every runtime version against every host and app combination.

That doesn't scale, and it's a little worse than an ordinary combinatorial problem - each axis belongs to a different team on its own release schedule, so nobody can hold the grid still long enough to fill it in.

FILL THE GRID

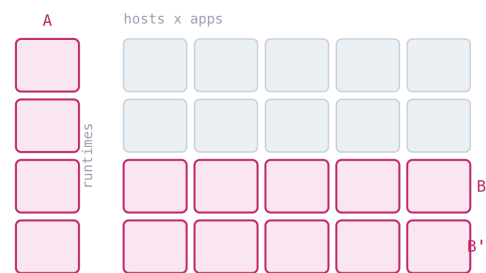
every runtime x every host and app



Twenty cells, and both axes keep moving.
No one team can hold it still.

TEST A COLUMN AND A ROW

one axis at a time, the other held still



A is a column we add - our own host, so any runtime can vary.
B and B' are two rows from one fork - the diff between them is the sign

Figure 1 - cutting the grid instead of filling it

A doesn't really sample the grid at all - it adds a column under a host we wrote ourselves, which leaves the runtime free to be any version. B and B' are two adjacent rows spawned from the same fork, differing only in the runtime, so comparing them is what tells us whether an upgrade reaches hosts and apps. Three lines rather than twenty cells.

The stack

- **Chains** hold state and enforce rules, with the ring-builder offchain worker and IPFS gateway inside that layer.
- The **device uniqueness backend** is the attester - the one genuinely separate off-chain service.
- **Hosts** implement TrUAPI (Triangle User-Agent Programming Interface) and run apps in sandboxes.
- **Apps** build against a TrUAPI version through product-sdk and ship as immutable DotNS bundles.

Because only the mobile hosts hold secret keys, the desktop and web are paired to one and lean on it for anything secret-dependent, like VRF proofs or signing.

It shows up in our own suites ([triangle-e2e](#)) - the Desktop and web runs both drive an Android emulator to approve signing.

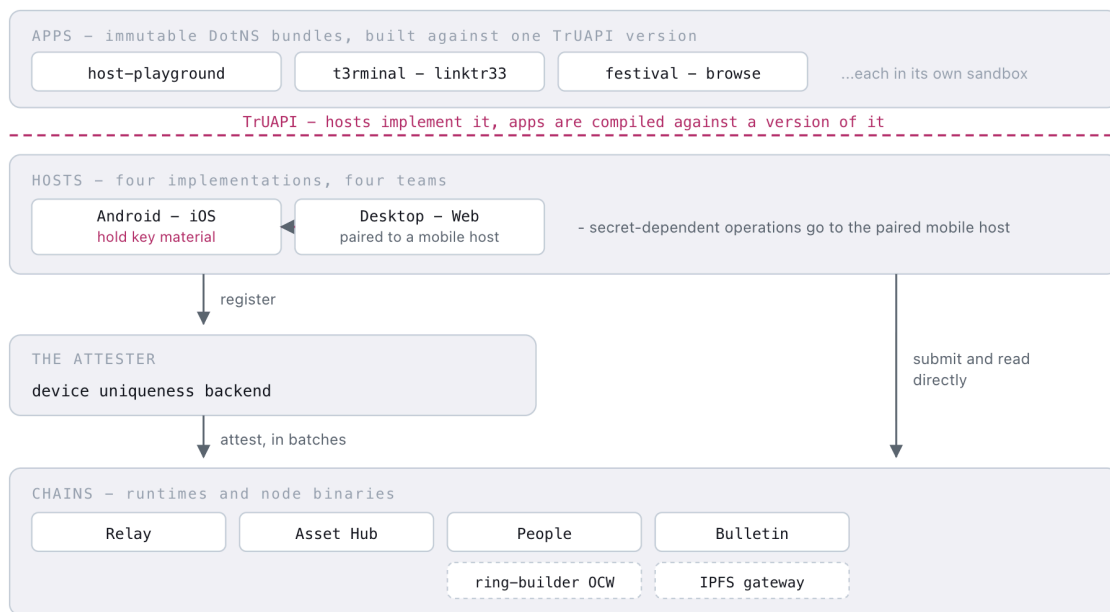


Figure 2 - the stack

Hosts reach the chains directly as well as through the attester, so a host change can break chain interaction with the attester nowhere in the path. Key custody sits only in the mobile hosts, which is part of why substituting "the host" means substituting more than one thing.

Why isolate

Chain runtimes enact on-chain, node binaries roll out to operators, the attester is a service deploy, product-sdk is a package release, each host ships through its own store, and apps go out as one-way DotNS deployments. None of those schedules are coordinated, and none are ours to set.

Thus there are two consequences:

Attribution

If a run uses every real component at whatever version it happens to be on, it only tells us about that one exact mix - and we didn't choose that mix. So when it fails, we learn the mix is broken, but not which change broke it.

Closing

When TrUAPI changes in a breaking way, host teams have to adopt it before apps can follow, and an app's DotNS bundle stays frozen against the product-sdk it was built with. During that window the four hosts are legitimately on different versions - so pairing a current app with a host that hasn't caught up yet tells us the window is open, not that anything is broken. And it could take days until all hosts are caught up.

Neither gets better by testing harder. Both get better if we hold the parts we're not asking about fixed.

Three configurations

The same environment definition can serve all three - what changes is how we treat each component: which ones are real and moving, which we supply ourselves, and which we pin.

A and **B** are the column and the row from Figure 1.

C it's the same machinery with nothing under test.

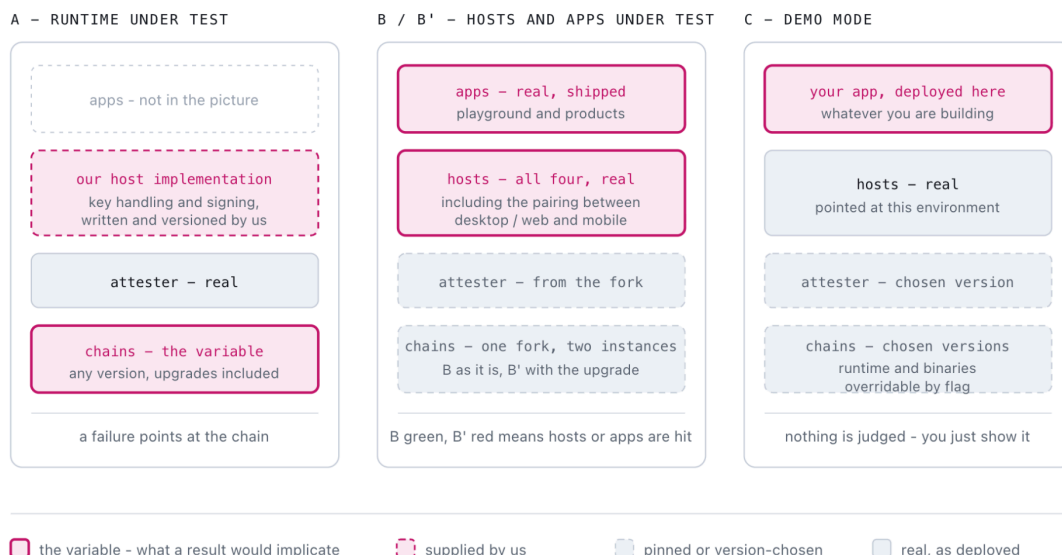


Figure 3 - one environment, three configurations

Each column pins or supplies everything except the thing it's asking about. A and B stay deliberately independent - if B started gating on a fresh runtime, a chain regression would show up as a host defect, and four host teams could end up hunting a bug they don't have.

A - Runtime and node testing

We supply the host side - key handling, proof generation, signing - as our own code, while the attester and chains stay real. Because everything above the chain is ours, a failure should point at the runtime or the binary, and the verdict doesn't have to wait on anyone's app release. This should provide a decent confidence level for proceeding with upgrade or recommending new binary version

B - Host and app testing

We fork the network we consider stable and treat that fork as the golden state. All four hosts are real, pairing included, and host-playground exercises the TrUAPI surface directly, so every host is measured against the same reference rather than against whatever a given product happens to call.

The comparison is the point. One bite gives us a single fork bundle, and we spawn two instances from it in parallel: **B** runs the app tests against the fork as it is, and **B'** applies the runtime upgrade first, then runs the same tests. Because both start from byte-identical state,

with the same host builds and the same deployed app versions, the runtime is the only difference between them.

Reading the pair is straightforward. **B** green and **B'** red means the upgrade affects hosts or apps. Both red tells us nothing, because something was already broken and there's no clean baseline to compare against. **B** red and **B'** green would be surprising enough to check by hand rather than trust.

This runs after **A**. If the runtime isn't healthy on its own, a red **B'** isn't telling us anything about hosts.

C - Demo mode

Spawn the whole stack at whatever versions you name, with chain runtime and node binaries overridable by flag. Deploy your app into it and open it in a real host. Nothing runs and nothing is judged - it's just a working Trinity you can hand to someone.

This covers things the test configurations don't:

- showing a feature to Gav,
- developing against a runtime that isn't deployed anywhere yet,
- reproducing a problem at exact versions,
- or running a workshop where everyone needs the same stack.

It's probably also the mode people would reach for most casually, which feels like an argument for making it the default entry point rather than something tucked inside a test harness.

What B and C need from the hosts

Mobile hosts hard-code two things at build time:

- the chain - genesis and RPCs -
- the device uniqueness backend, which is itself paired to that chain.

That's why Previewnet and Paseo Next v2 each need their own build today.

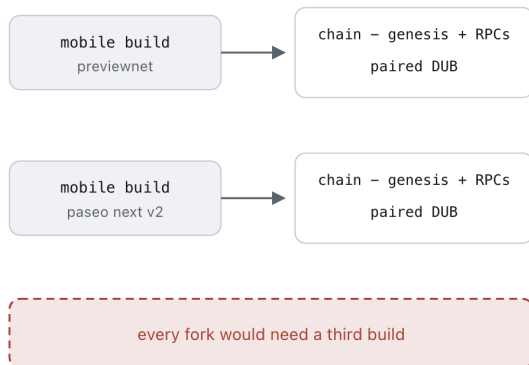
Mobile also isn't optional here - key custody sits only there, so every app test needs a mobile host in the loop, even when the surface we're actually testing is Desktop or web.

Taken together, that's what makes **B** and **C** hard today.

Both want a freshly spawned environment, and a new environment would mean a new host build - which host build and release times don't really allow.

What would help is resolving endpoints at startup rather than at compile time.

TODAY - ONE BUILD PER NETWORK



RUNTIME OVERRIDE - ONE BUILD

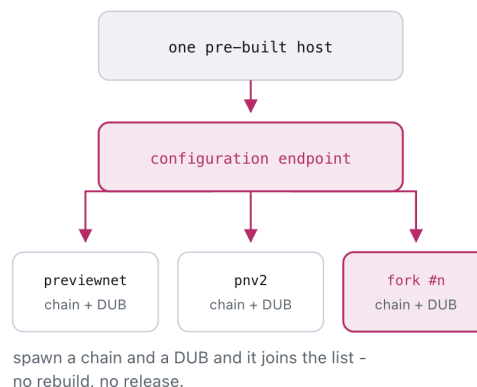


Figure 4 - the change B and C depend on

The endpoints move from compile time to startup. Web already works this way - [dotli-community#156](#) ships prebuilt bundles repointed at run time via `$DOTLI_NETWORK`. Its override covers endpoints only (`label`, `rpcs`, `ipfsGateways`); `genesis` and `dotns` stay fixed on purpose, since a repointable DotNS registry would redirect every `.dot` lookup - so a fork with a new genesis would need more than what shipped

Web being done doesn't quite unblock **B** on its own, though, because web still needs pairing. So at least one mobile host would need to become configurable too.

That difference also shapes where each configuration can run.

A needs none of this - chain overrides plus chain-tests, which we can keep in CI and offline.

B and **C** need real infrastructure:

- the stack spawned in the cloud, exposed with certificates etc
- a configuration endpoint listing what's available
- A pre-built mobile app could then hook into any of them, and the same mechanism would probably serve Web and Desktop - giving us one way to point any host at any environment.

Whether that list eventually hardens into a proper manifest for the stack is a separate question, and not something we need to decide here.

Notes from meeting:

Currently the apps are built with a *hardcoded environment* (paseo-next-v2) and connect through its genesis. This constraint makes the current app release incompatible with the EM design since we will need a new build per environment instance.

It is expected that in the future the app itself will have the logic to select the environment configuration at bootstrap (allowing to dynamically build a list of environment instances to use from an indirection call).

Following the current constraint, we will focus on testing the runtime upgrades in the environment instances (e.g paseo like environments) to ensure a rapid lifecycling if the runtime releases and keep paso-next-v2 updated.

[WIP] Design: Environment Manager

[wip] Environment Manager

Author: Nikola Djoric Javier Viola Villanueva

Last Edited: 12 Aug 2026

2026 Q3 MVP

Environment Schema and Storage

The environment schema will use a layer of configuration and customizable components:

Components

Each component allows us to represent a *deployable* unit, with a well defined **version** and **location** (repository/image) and extra fields to allow customization of each one.

Program

Represents a *long running* program (e.g: a client/node binary)

Fields:

- name: binary name (e.g. polkadot)
- version: version to use (e.g. stable2606, should match the release tag)
- repo_url: url of the repository
- image_url: url of the image (e.g. docker.io/parity/polkadot, the tag should match the version)
- extra_bins[]: Allow to specify extra binaries to fetch (e.g. workers)
- extra_args[]: Extra arguments as array of strings to pass to the binary
- local_path: Optional, allow to specify a local path to override the location to use.

Runtime

Represents the *runtime* to use in a particular chain (e.g: the wasm file).

Fields:

- name: release asset name (e.g. westend_runtime-v1024001)
- version: version to use (e.g. stable2606, should match the release tag)
- repo_url: url of the repository
- preset: Genesis preset to build the chain spec with, e.g. `development`.
- local_path: Optional, allow to specify a local path to override the location to use.

Script

Represents a script to be executed only one time.

Fields:

- name: binary name (e.g. polkadot)
- version: version to use (e.g. stable2606, should match the release tag)
- repo_url: url of the repository
- repo_path: Path to the script within the repository.
- extra_args[]: Extra arguments as array of strings to pass to the binary
- local_path: Optional, allow to specify a local path to override the location to use.

Environment top level schema

The environment is constructed with the following shape:

- **name**: Used as id
- **type**: (discriminant used to spawn from fork / genesis)
- **Relaychain**
 - chain: Chain to use (e.g. westend)
 - program: [Program](#)
 - runtime: [Runtime](#)
- **Parachains []**
 - chain: String // Chain to use (e.g asset-hub)
 - para_id: Number // Parachain id
 - extra_cores: Number // Each parachain receives 1 core automatically, here we set the extra ones.
 - program: [Program](#)
 - runtime: [Runtime](#)
- **Satellites []**
 - [Program](#) | [Script](#)

Example of environment definition:

JSON

```
{
  "name": "Westend-fork",
  "type": "fork",
  "relaychain": {
    "chain": "westend",
    "program": {
      "name": "polkadot",
      "version": "polkadot-stable2606",
```

```

    "repo_url": "github.com/paritytech/polkadot-sdk",
    "image_url": "https://hub.docker.com/r/parity/polkadot",
    "extra_bins": [
        "polkadot-execute-worker",
        "polkadot-prepare-worker"
    ]
  },
  "runtime": {
    "name": "westend_runtime-v1024001",
    "version": "polkadot-stable2606",
    "repo_url": "github.com/paritytech/polkadot-sdk"
  }
},
"parachains": [
  {
    "chain": "asset-hub",
    "program": {
      "name": "polkadot-parachain",
      "version": "polkadot-stable2606",
      "repo_url": "github.com/paritytech/polkadot-sdk",
      "image_url": "https://hub.docker.com/r/parity/polkadot"
    },
    "runtime": {
      "name": "asset-hub-westend_runtime-v1024001",
      "version": "polkadot-stable2606",
      "repo_url": "github.com/paritytech/polkadot-sdk"
    },
    "para_id": 1000
  }
],
"satellites": [
]
}

```

This allows us to design a declarative layer, easy to create / update for developers and extensible if needed. From the declaration we can *resolve, validate and derive* the **network definition** to emit and use to spawn with *zombienet*.

This process will be one way direction and we can map an environment declaration to a network definition expanded (and store both in the repo database).

Wiring

We will use the same *wiring logic* provided by *zombienet* (using the `{{ZOMBIE:node:filed}}` pattern) to allow users to *wire* components (e.g. satellites with node info).

Emitting

The environment manager will internally plan and emit a *zombienet network definition file* (***toml***), which will be *stored* alongside with the environment definition and will be used to spawn instances of the environment.

Lifecycle

The system will allow users to create environments (use custom components or ones available in a catalog), and from these environments *spawn instances*. This will be the ones the users interact with and can override.

A normal flow could be, define an environment (e.g. staging) with the latest version of the components available in the catalog. Once a defined and *stored* user can spawn multiple instances of this defined environment and even upgrade some of the components (runtime/nodes) in-place in the instance.

Output

Each instance created by the EM will expose the running configuration and the available endpoints to interact directly with the components (e.g. rpc/ws endpoints, metrics endpoints). This information will be presented in a **json** format to easily be consumed by scripts/test-runners.

Actions

The EM will also allow it to perform some well defined actions in the environment, like:

- Changing the binary that a node is running.
- Add / remove nodes.
- Pause / resume nodes.

This action should be well defined and be explicit in terms of which components support them.

REST API

- `/api/v1:`
 - GET
 - `/version`: SemVer, build date, maybe git hash
 - `/environments`: list all known environments from Github
 - `/environments/ID`: info for this specific environment. List all instances of it (forks) and default component versions.
 - `/instances/ID`: Info on running instance. Exposed the environment ID and the running component endpoints.
 - `/instances/ID/component/ID`: Status on a component inside a running instance. Should return component version, uptime and RPCs.
 - POST
 - `/instances/ENV_ID` (body: Overrides, new_env: bool), (return: Instance ID)
 - Spawn a new instance of an existing environment config with overrides
 - If new_env is true, then create a new environment definition
 - Returns the new instance Id immediately.
 - `/environments/ENV_ID` (body: Environment)
 - Creates a new environment definition with the parameters passed in the body
 - This results in a PR in the GH environment repository.
 -
 - DELETE
 - `/instance/ID` (body: Instance ID, retention-mode=delete | retain-30d)
 - Schedules the instance for shutdown.
 - The retention enum can be used to either hard-delete an instance or keep its state persisted for future inspection.
 -

[WIP] Design: Omni Tester

Overview

Abstract

The omni-tester is a *component* that acts as liaison between the Environment Manager and the end-user e2e test. In a way, wrap optionally the *spawning phase* and run the desired test suite on top of the running network.

This component also interacts with github, in order to pull test-suite/s and create pr/issues according to the observed result.

Components

Rest API

Provide a unified interface to interact with the component, allow users to run test by providing a schema defined input with:

- Test suite definition
- Environment definition
- Reporting options

Github Integration

We can listen to webhooks to act (run test/reports) on demand or we can also support schedule tests (cron like).

User Flows

On demand e2e test through rest api

This case cover the direct interaction between the end-user and the component, and is initiated by the user through an api request (could be using some UI or cli), with all the input

information to run the e2e test on the desired environment (Note, we can use an already running environment or spawn a new one).

The component will observe the result and report it with all the artefacts to easily debug and make diagnostic.

New release candidate through github integration

The component will also *listen to new* candidate releases (runtime/client) and act according to the scheduling logic. In this case we should have set a e2e suite attached to the candidate release and the component will spawn a new environment to run it on top.

The report of the observed result should follow the same attached logic of the test suite. (e.g open issues in repo / send msg to element)

Scheduled e2e

We want to run some e2e tests on scheduling based, and not on top of new candidates in order to detect regression (bugs/performance). This particular flow will need some extra steps or logic in order to get the latest version (not yet available in candidates) of the runtime/clients.